

UT 2 COMPONENTES REUTILIZABLES

En esta unidad de trabajo aprenderás qué es una interfaz gráfica y los fundamentos básicos de Flutter, un framework para crear interfaces de aplicaciones.

Índice

1.1 Creación de componentes reutilizables.....1
1.2 Nuestro primer widget propio.....1
1.3 Organizar los componentes en archivos.....5

2. COMPONENTES REUTILIZABLES

1.1 CREACIÓN DE COMPONENTES REUTILIZABLES

Un componente reutilizable es una parte de la interfaz que encapsula una estructura o comportamiento y que podemos utilizar en distintos lugares sin tener que escribir de nuevo su implementación.

Sus principales ventajas son:

- Reutilización: escribimos el código una vez y lo utilizamos muchas veces.
- Mantenimiento: los cambios se realizan en un único lugar.
- Legibilidad: la interfaz principal resulta más sencilla de comprender.
- Consistencia: los elementos similares mantienen el mismo aspecto y comportamiento.
- Pruebas: podemos comprobar el funcionamiento de cada componente por separado.

1.2 NUESTRO PRIMER WIDGET PROPIO

Vamos a comenzar con un ejemplo sencillo: una tarjeta que muestra un mensaje informativo.

Primero, observemos cómo sería la interfaz sin crear ningún componente personalizado.

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MiAplicacion());
}

class MiAplicacion extends StatelessWidget {
  const MiAplicacion({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Mis avisos'),
        ),
        body: Center(
          child: Card(
            child: Padding(
              padding: const EdgeInsets.all(16),
              child: Row(
                mainAxisSize: MainAxisSize.min,
                children: const [
                  Icon(Icons.info, color: Colors.blue),
                  SizedBox(width: 12),
                  Text('Recuerda guardar los cambios'),
                ],
              ),
```

```
          ),
        ),
      ),
    ),
  );
 }
}
```

Este código funciona, pero la estructura de la tarjeta está escrita directamente dentro de la pantalla principal.

Vamos a extraerla a un widget propio.

> **¡¡UTILIZA REFACTOR!! Extract Widget-> NewWidget: TarjetaAviso**

```
class TarjetaAviso extends StatelessWidget {
 const TarjetaAviso({super.key});

 @override
 Widget build(BuildContext context) {
  return Card(
   child: Padding(
    padding: const EdgeInsets.all(16),
    child: Row(
     mainAxisSize: MainAxisSize.min,
     children: const [
      Icon(Icons.info, color: Colors.blue),
      SizedBox(width: 12),
      Text('Recuerda guardar los cambios'),
     ],
    ),
   ),
  );
 }
}
```

Aquí aparece una cosa nueva que no habíamos visto hasta ahora que es el **constructor**.

```
const TarjetaAviso({super.key});
```

El constructor permite crear instancias de nuestro widget.

La expresión `super.key` permite recibir y transmitir una clave opcional al widget padre. Las claves ayudan a Flutter a identificar widgets cuando reconstruye la interfaz.

Ahora podríamos llamar a ese widget varias veces desde main, pero siempre escribiría el mismo mensaje. Para mostrar mensajes diferentes, tenemos que trabajar con **propiedades**, **constructores** y **parámetros**.

Una propiedad es un atributo de una clase que almacena información sobre sus objetos.

Por ejemplo, una tarjeta de aviso podría tener las siguientes propiedades:

Propiedad	Tipo	Ejemplo
`mensaje`	`String`	`'Guarda los cambios'`
`icono`	`IconData`	`Icons.warning`
`color`	`Color`	`Colors.orange`

Estas propiedades nos permitirán personalizar cada instancia del componente.

En Dart, las declaramos dentro de la clase:

```
class TarjetaAviso extends StatelessWidget {
  final String mensaje;
  final IconData icono;
  final Color color;

  // Constructor y método build...
}
```

La palabra `final` indica que una variable solo puede recibir un valor una vez.

En los widgets de Flutter es habitual declarar las propiedades como `final`, porque los widgets son objetos inmutables: una vez construida una instancia, sus propiedades no se modifican.

¿Qué diferencia hay entre `final` y `const`?

- `final`: el valor se asigna una sola vez, incluso si se conoce durante la ejecución.
- `const`: el valor debe ser una constante conocida en tiempo de compilación.

Un constructor es un mecanismo que permite inicializar un objeto cuando lo creamos.

En nuestro componente, el constructor recibirá los valores de las propiedades.

```
class TarjetaAviso extends StatelessWidget {
  final String mensaje;

  const TarjetaAviso({
    super.key,
    required this.mensaje,
  });

  // Método build...
}
```

`TarjetaAviso`: el constructor tiene el mismo nombre que la clase.

- `{ }`: indican que los parámetros son nombrados.
- `super.key`: permite recibir una clave opcional para el widget.
- `required`: indica que el parámetro es obligatorio.
- `this.mensaje`: asigna el valor recibido a la propiedad `mensaje` del objeto.
- `const`: permite construir instancias constantes cuando sus argumentos también son constantes.

Ahora, si escribimos:

```
const TarjetaAviso(mensaje: 'Guarda los cambios')
```

El parámetro mensaje es obligatorio. Si queremos que sea opcional, podemos asignarle un valor por defecto:

```
const TarjetaAviso({
  super.key,
  this.mensaje = 'Sin avisos',
});
```

Podemos añadir más propiedades: icon y su color.

```
import 'package:flutter/material.dart';

class TarjetaAviso extends StatelessWidget {
  final String mensaje;
  final IconData icono;
  final Color color;

  const TarjetaAviso({
    super.key,
    required this.mensaje,
    this.icono = Icons.info,
    this.color = Colors.blue,
  });

  @override
  Widget build(BuildContext context) {
    return Card(
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Row(
          mainAxisSize: MainAxisSize.min,
          children: [
            Icon(icono, color: color),
            const SizedBox(width: 12),
            Flexible(
              child: Text(mensaje),
            ),
          ],
        ),
      ),
    );
  }
}
```

Aquí aparece `Flexible`, un widget que permite que el texto utilice el espacio disponible dentro del `Row` y se adapte cuando el mensaje es largo. No es necesario dominarlo todavía.

Si observas el constructor, verás que la propiedad mensaje es obligatoria, pero icono y color no lo son, por lo que podemos llamar al constructor de varias maneras:

const TarjetaAviso(mensaje: 'Información general',)	const TarjetaAviso(mensaje: 'Cuidado con cambios', icono: Icons.warning, color: Colors.orange,)	const TarjetaAviso(mensaje: 'Operación ok', icono: Icons.check_circle, color: Colors.green,)

Las propiedades de un componente pueden ser de cualquier tipo: String, int, double, bool, Color, IconData, Widget.

1.3 ORGANIZAR LOS COMPONENTES EN ARCHIVOS

Aunque podemos definir todos nuestros widgets dentro de `main.dart`, no es una buena práctica cuando la aplicación comienza a crecer.

Una aplicación con decenas de componentes resultaría difícil de mantener si todo estuviera en un único archivo.

Una estructura sencilla podría ser:

```
lib/
├── main.dart
├── screens/
│   └── pantalla_avisos.dart
└── widgets/
    └── tarjeta_aviso.dart
```

Cada archivo tiene una responsabilidad:

- `main.dart`: punto de entrada y configuración principal.
- `screens/pantalla_avisos.dart`: pantalla completa de la aplicación.
- `widgets/tarjeta_aviso.dart`: componente reutilizable.

Esta organización no es obligatoria en Flutter, pero nos ayuda a mantener el proyecto ordenado.

1. Crear el archivo del componente

   Dentro de `lib`, creamos una carpeta llamada `widgets`.

   En ella añadimos el archivo `tarjeta_aviso.dart`:

2. Importar el componente:

   Para utilizarlo desde `main.dart`, debemos importar el archivo donde lo hemos definido.

   Si el proyecto se llama `mi_aplicacion`, podemos escribir:

```
import 'package:mi_aplicacion/widgets/tarjeta_aviso.dart';
```

A partir de ese momento, podemos utilizar `TarjetaAviso` desde `main.dart` o desde cualquier otro archivo que lo importe.

Esta separación nos permite modificar el diseño de la tarjeta sin tener que buscar su implementación dentro de la pantalla principal.